07
Advanced mechanisms of the Scala language

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Magistrale in Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2025/2026

Outline

1. Aliases, abstract types, and abstract modelling
2. Traits and mixins
3. Exports and delegation vs inheritance
4. Variance and generics
5. Extension methods
6. Contextual programming with given instances
7. ScalaTest
8. For comprehension

Abstract types (and aliases)

Mechanism

- a trait or abstract class can include the definition of an abstract type of the form "`type T`", or "`type T<:R`" (where `R` is a concrete type)
- when subclassed, such a type needs to be concretised to an actual type before creating objects, writing `type T=R` (where `R` is another type)
- it is a form of type parameterization, where the abstracted types are not listed directly in the parameterized type, but as members
- the abstract type can be generic, e.g. as in `type MySeq[T] = List[T]`

Applications

1. to create aliases, possibly abstract and then concretised in subclasses
   - like the aliases created in `scala.Predef` (`Function`, `Map`, `Set`, ...)
   - in this case, it is essentially a mechanism similar to import renaming, though it can be used with advanced types, such as generics
2. to support abstract modelling of concepts (like in ADTs)
3. to work with path-dependent types, supporting advanced patterns like family polymorphism.. will discuss at the end of the course

Aliases example

```scala
// an extract of scala.Predef
class Predef:
  type Class[T] = java.lang.Class[T]
  type Function[-A, +B] = (A) => B
  type Map[A, +B] = collection.immutable.Map[A, B]
  type Set[A] = collection.immutable.Set[A]
  type String = java.lang.String

// aliases and corresponding constructors
object MyAliases:
  type MySeq[T] = collection.immutable.Vector[T]
  def MySeq[T](elems: T*) = collection.immutable.Vector(elems)
  type MySet[T] = collection.immutable.HashSet[T]
  def MySet[T](elems: T*) = collection.immutable.HashSet(elems)
  type MyMap[A, +B] = collection.immutable.HashMap[A, B]
  def MyMap[A, B](elems: (A, B)*) = collection.immutable.HashSet(elems)

object Aliases extends App :
  import MyAliases.* // using aliases
  val seq = MySeq(10, 20, 30)
  println(s"$seq,${seq.getClass}")
  val set = MySet(10, 20, 30)
  println(s"$set,${set.getClass}")
  val map = MyMap(10 -> "a", 20 -> "b", 30 -> "c")
  println(s"$map,${map.getClass}")
```

A case study: Exams, abstract modelling

```scala
trait Exams:
  // abstracted types
  type Student
  type Teacher
  type Course
  type Evaluation
  trait Session:
    def id: Int

  // defined types
  case class Exam(course: Course, session: Session, student: Student)
  type Registration = (Exam, Evaluation)

  // methods
  def register(registration: Registration): Unit
  def evaluation(student: Student, course: Course): Evaluation

abstract class AbstractExams extends Exams:
  private var registrations: Set[Registration] = Set()

  override def register(registration: Registration) =
    registrations = registrations + registration

  override def evaluation(student: Student, course: Course) =
    registrations
      .collect:
        case (Exam(`course`,call,`student`), eval) => (call.id,eval)
      .maxBy(_._1)._2
```

A case study: Exams, implementation and use

```
class BasicExamsImpl extends AbstractExams:
  override type Student = String
  override type Teacher = String
  override type Course = String
  override type Evaluation = Int

  case class SessionWithId(id: Int) extends Session
  def session(id: Int): Session = SessionWithId(id)

object WorkWithExams extends App:

  val exams = new BasicExamsImpl()
  exams.register((exams.Exam("oop",exams.session(0),"mirko"),20))
  exams.register((exams.Exam("oop",exams.session(1),"mirko"),25))
  exams.register((exams.Exam("sisop",exams.session(0),"mirko"),30))

  println(exams.evaluation("mirko","oop")) // 25
  println(exams.evaluation("mirko","sisop")) // 30
```

On such abstract modelling technique

A quintessential technique

- can define a whole "system" by a trait of concepts
- can refine it progressively, making it more and more complete
- this is THE approach to address complex abstractions
- cannot be easily achieved by other mainstream languages

Additional techniques that will help

- mixin-technique will be used to mix many "slices"
- self-types will be used to define slices with expectations without inheritance
- family polymorphism will be used to support mutual dependencies

Outline

1. Aliases, abstract types, and abstract modelling
2. Traits and mixins
3. Exports and delegation vs inheritance
4. Variance and generics
5. Extension methods
6. Contextual programming with given instances
7. ScalaTest
8. For comprehension

Traits vs abstract classes

Abstract classes

- basically identical to abstract classes in Java
- directly translated to JVM's abstract classes
- like classes, can be extended in a single way
- they pose no significant performance delay or burden wrt Java's

Traits

- interfaces with also implementation (and hence, methods, fields, . . .)
- translated to interfaces with additional static methods
- the class extending (multiple) traits calls the static methods as needed
- traits "linearization" is used to stack super-traits/classes from left(up) to right(down), creating a sequence of decorations
- they may lead to some performance delay—again, typically don't care

Main application for traits

Abstract concepts

- like for Java interfaces, especially when providing only methods

Rich interfaces

- interfaces for many different implementations
- providing few abstract methods
- additionally giving many template methods
- notable example: `Iterable`

Interface for strategies, and type classes

- for instance, the `Ordered[T]` trait
- but also any `FunctionX[..]`, namely, any functional type like `(Int,Double)=>String`

Class templates

- partially complete classes, to be multiply extended

Stackable modifications – the key mixin property

- modification to a class, to which this trait is mixed-in "later"

Rich interfaces

```scala
case class Point(x: Int, y: Int)

trait Rectangular:
  // abstract methods
  def topLeft: Point
  def bottomRight: Point
  // template methods
  def left: Int = topLeft.x
  def right: Int = bottomRight.x
  def top: Int = topLeft.y
  def bottom: Int = bottomRight.y

case class Rectangle(
  override val topLeft: Point,
  override val bottomRight:Point) extends Rectangular

object TryRichInterface extends App:
  val r = Rectangle(Point(0,0),Point(10,20))
  println(s"${r.left},${r.right}")
  println(s"${r.top},${r.bottom}")
```

Another example of rich interface: scala.Ordered trait

```scala
trait Ordered[T] extends Comparable[T]:
  def compareTo(that: T): Int
  def <(that: T): Boolean = (this compareTo that) < 0
  def >(that: T): Boolean = (this compareTo that) > 0
  def <=(that: T): Boolean = (this compareTo that) <= 0
  def >=(that: T): Boolean = (this compareTo that) >= 0

case class MyInt(n:Int) extends Ordered[MyInt]:
  override def compareTo(that: MyInt) = this.n - that.n

object TryOrdered extends App:
  println( MyInt(5) < MyInt(7))
  println( MyInt(5) <= MyInt(7))
  println( MyInt(5) > MyInt(7))
  println( MyInt(5) >= MyInt(7))

// package scala has Ordered[T] (for Comparable)
// package scala has Ordering[T] (for Comparator)
```

Mixins in Scala

Mixin

A mixin is a unit of code, possibly partially implemented, that can be used (along with other mixins) to form a class: such a class is required to provide the missing implementations of all extended mixins. Mixins are sometimes described as being "included" rather than "inherited", or also, being "stacked in": generally, they do not imply substitutability, just reuse.

Mixins in Scala

- Scala traits can be used as mixins
- A class can extend with multiple Scala traits
- The super operator behaves especially in traits, allowing traits to be chained using composition in addition to inheritance

Showcasing mixins: Parser

```scala
abstract class Parser[T]:
  def parse(t: T): Boolean  // accept the token! did it work?
  def end: Boolean        // tokens are over! is this ok?
  def parseAll(seq: Seq[T]): Boolean = (seq forall (parse(_))) & end // could use & or &&

// A parser consuming any sequence of elements in `chars`
case class BasicParser(chars: Set[Char]) extends Parser[Char]:
  override def parse(t: Char) = chars contains t
  override def end = true

// A mixin rejecting empty strings (see: a sort of decorator!)
trait NonEmpty[T] extends Parser[T]:
  private var empty = true
  abstract override def parse(t: T) = {empty = false; super.parse(t)} // who is super??
  abstract override def end = !empty && super.end

class NonEmptyBasicParser(chars: Set[Char]) extends BasicParser(chars) with NonEmpty[Char]

object TryParsers extends App:
  println(BasicParser(Set('a','b','c')).parseAll("aabc".toList)) // true
  println(BasicParser(Set('a','b','c')).parseAll("aabcdc".toList)) // false
  println(BasicParser(Set('a','b','c')).parseAll("".toList)) // true

  // Note NonEmpty being "stacked" on to a concrete class
  // Bottom-up decorations: NonEmptyParser -> NonEmpty -> BasicParser -> Parser
  println(NonEmptyBasicParser(Set('0','1')).parseAll("0101".toList)) // true
  println(NonEmptyBasicParser(Set('0','1')).parseAll("0123".toList)) // false
  println(NonEmptyBasicParser(Set('0','1')).parseAll(List())) // false
```

Parser explanations

Note the nice design

- `Parser` as an abstract class with (functional) template method
- `BasicParser` as a baseline implementation
- `NonEmpty` as a mixable decoration, just checking non-emptyness
- note stacking of `NonEmpty` on top of `BasicParser`, to create `NonEmptyParser`
- note this a multiple-inheritance support (clause `with`)
- note `NonEmpty` refers to an unknown `super`—nothing similar exists in Java

Trait (mixin) linearization

Trait linearization in the `Parser` example

How is multiple inheritance with traits rendered in the JVM? By decoration:

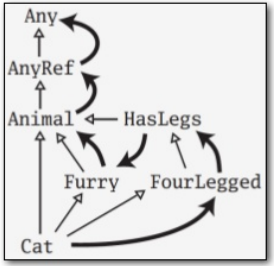
- recall that decoration is inheritance + delegation
- An object of `NonEmptyParser` decorates a `NonEmpty` which decorates a `BasicParser` which extends `Parser`
- In `NonEmptyParser`, `NonEmpty` will use an instance of `BasicParser` as `super`
- Not really important when programming: "things just work"
- Just note that linearization never includes a class twice

Trait linearization (see Odersky&Spoon&Venners' book)

Linearly Ordering Traits

a class is always linearized before all of its superclasses and mixed in traits

```
class Animal
trait Furry extends Animal
trait HasLegs extends Animal
trait FourLegged extends HasLegs
class Cat extends Animal with Furry with FourLegged
```



Type	Linearization
Animal	Animal, AnyRef, Any
Furry	Furry, Animal, AnyRef, Any
FourLegged	FourLegged, HasLegs, Animal, AnyRef, Any
HasLegs	HasLegs, Animal, AnyRef, Any
Cat	Cat, FourLegged, HasLegs, Furry, Animal, AnyRef, Any

Mixins with self-types: from inheritance to decoration

Syntax

```
trait T ...  { <name>:<type> => ...  }
```

Meaning

- to instantiate T one needs it to be mixed-in with a class/trait with type <type>
- name is a reference equal to this, but given type <type>: it will be the object that one will create by mixin
- basically, it supports a form of inheritance without subtyping, internally achieved by delegation/decoration
- creating mere "decorations" is a typical application

A particular case..

- if :<type> is omitted in the syntax, then name is given the type of the class in which it is used, and it is just used as alias for this, to be used by inner classes to avoid clash of the many visible this

Mixins with self-types: from inheritance to decoration

```scala
trait Producer:
  def get(): String

// not a subtype of Producer, but truly a mixin!
trait ProducerUtils:
  producer: Producer =>
  def getMany(n: Int): Seq[String] =
     for i <- 0 to n yield producer.get()

class RandomProducer extends Producer:
  override def get(): String = Math.random().toString

class ConstantProducer(s:String) extends Producer:
  override def get(): String = s

class FullRandomProducer extends RandomProducer with ProducerUtils

@main def tryProducers =
  // attaching utils to any Producer
  val producer = new FullRandomProducer
  val producer2 = new RandomProducer() with ProducerUtils
  val producer3 = new ConstantProducer("a") with ProducerUtils
```

self => notation for this

```
trait FunctionOption[-A, +B]:
  self =>
  def apply(a: A): Option[B]

  final def andThen[C](f: FunctionOption[B,C]): FunctionOption[A,C] =
    new FunctionOption[A,C]:
      override def apply(a: A): Option[C] =
        self.apply(a).flatMap(b => f.apply(b))
        //this.apply(a).flatMap(b => f.apply(b)) // this would not compile

@main def mainFunction =
  val f: FunctionOption[Int, Int] =
    n => Some(n) filter (_ % 2 == 0) map (_/2)
  val g: FunctionOption[Int, Int] =
    n => Some(n+1)

  val h: FunctionOption[Int, Int] = f andThen g
  println(h(5)) // None
  println(h(6)) // Some(4)
```

Outline

1. Aliases, abstract types, and abstract modelling
2. Traits and mixins
3. Exports and delegation vs inheritance
4. Variance and generics
5. Extension methods
6. Contextual programming with given instances
7. ScalaTest
8. For comprehension

Inheritance vs delegation, and Scala exports

A modern recipe: Consider delegation instead of inheritance!

- least power principle
- inheritance opens a class to subclasses too much, and hardly deals with multiple reuses
- substitutability is better achieved by interfaces/traits alone, focussing on contract
- delegation, in conjunction with refined access modifiers, is preferable

$\Rightarrow$ the problem with delegation is boilerplate code for forwarders

Scala 3 exports

- to be used in a class/object, almost the same syntax of imports
- an export directly transfers visibility of stable members "content" to outside
- coherently, exports can also be used to create units of import

Exports at work

```
trait Counter:
  def value: Int
  def increment(): Unit

class BasicCounter extends Counter:
  private[Exports] var counterValue = 0
  def value: Int = counterValue
  def increment(): Unit = counterValue = counterValue + 1
  def reset(): Unit = counterValue = 0

class MultiCounter extends Counter: // no inheritance, just substitutability
  private val counter = new BasicCounter() // no inheritance, just delegation
  export counter.*  // export value, increment, reset
  def decrement(): Unit = counterValue -= 1

class FancyCounter extends Counter: // no inheritance, just substitutability
  private val counter = new BasicCounter() // no inheritance, just delegation
  private val initDate = java.util.Calendar.getInstance();
  export counter.{reset as _, *}  // export all but reset
  export initDate.getTime as creationTime // export with alias
  //def creationTime(): String = initDate.toString

val c: Counter = new FancyCounter() // no inheritance, just substitutability
println(c.value)
println(c.increment())
val c2 = new FancyCounter()
// println(c2.decrement()) // won't work
println(c2.creationTime())
```

Importing(/Exporting) units

```scala
object MyStandardExports:
  export scala.collection.mutable.{
          ListBuffer => StandardList,
          HashSet => StandardSet,
          HashMap => StandardMap
  }
  export scala.Array.*
  export java.lang.Math.*
  export java.util.Date

@main def tryExports =
  import MyStandardExports.*

  println(StandardSet(10,20,30).getClass) // using mutable.HashSet alias
  println(sin(PI / 2)) // using Math.*
  println(concat(Array(10, 20), Array(30, 40)).toList) // using Array.*
  println(new Date()) // using java.util.Date
```

Outline

1. Aliases, abstract types, and abstract modelling
2. Traits and mixins
3. Exports and delegation vs inheritance
4. Variance and generics
5. Extension methods
6. Contextual programming with given instances
7. ScalaTest
8. For comprehension

Type parameterization (generics)

Main facts

- classes, traits, types and methods, can come with type parameters
- they can be used in the definition as they were types
- at the call side, the type parameter is (almost) often inferred
- they could be "reified", but will see how later

Advanced mechanism: declaration-site variance

- Scala has so-called definition-site variance (wrt use-site variance of Java wildcards) – see Igarashi&Viroli at ECOOP 2002
- can declare a type parameter `A` as co-variant (`+A`) or contra-variant (`-A`)
- the compiler checks if this makes sense (as of Liskov's substitution principle)
- e.g. with co-variant lists `List[+T]`, we have: `List[Int]<:List[AnyVal]` – setters would be disallowed though, due to contra-variant position
- in libraries, see e.g. `Function2[-T1, -T2, +R]`

Advanced mechanism: upper-lower bounds

- upperbounds (`[A<:UpperBound]`) or lowerbounds (`[A>:LowerBound]`) or both, can be put to constraint the possible instantiations of a type parameter

Variant definitions in Scala library

```scala
object Variance extends App:

  /*
  package scala.*
  abstract class Option[+A]
  trait PartialFunction[-T1, +R] // { case x => ... }
  trait Function0[+R]        // ()=>R
  trait Function1[-T1, +R]   // (T1)=>R
  trait Function2[-T1, -T2, +R] // (T1,T2)=>R
  case class Tuple2[+T1, +T2](_1: T1, _2: T2) // (T1,T2)

  package scala.collection
  trait Iterator[+A]
  trait Set[A] // ... extends (A) => Boolean
  trait Seq[+A] extends PartialFunction[Int, A]... extends (Int) => A
  trait Map[K, +V] extends PartialFunction[K, V]... extends Iterable[(K,V)]
  */

  val v: Option[List[Any]] = Some[List[Int]](List(10,20,30))
  val w: Option[Iterable[Any]] = Some[Set[Int]](Set(10,20,30))
  val f: String => Any = (s:Any) => s.toString
  val s: String => Boolean = Set("a", "b", "c")
  val seq: Int => String = List("a", "b", "c")
  val m: String => Int = Map( "a" -> 1, "b" -> 2, "c" -> 3)
```

An example: Stack[T]

```scala
trait Stack[T]:
  def top: Option[T]
  def pop: Option[(T,Stack[T])]
  def push(t: T): Stack[T]
  def toList: List[T]

object Stack:

  def apply[T](elems: T*): Stack[T] = StackImpl(elems.toList)

  private class StackImpl[T](val list: List[T]) extends Stack[T]:
    override def top = list.headOption
    override def pop = top map ((_,StackImpl(list.tail)))
    override def push(t: T) = StackImpl(t :: list)
    override def toList = list

object TryStacks extends App:
  var stack = Stack(10)
  stack = stack push 20
  println(stack.top) // Some(20)
  stack = stack.pop.get._2 //[10]
  println(stack.top) // Some(10)
  stack = stack.pop.get._2 //[]
  println(stack.top) // None

  // COVARIANCE PROBLEM
  //var generalStack: Stack[Any] = Stack[Int](10,20,30)
  //generalStack = generalStack.push("a string") // this would corrupt the stack
```

Making it covariant: CoStack[+T]

```scala
trait CoStack[+T]: // a covariant stack
  def top: Option[T]
  def pop: Option[(T,CoStack[T])]
  def push[R >: T](r: R): CoStack[R]  // a more general push is needed
  def toList: List[T]

// same implementation of Stack
object CoStack:

  def apply[T](elems: T*): CoStack[T] = CoStackImpl(elems.toList)

  private case class CoStackImpl[+T](list: List[T]) extends CoStack[T]:
    override def top = list.headOption
    override def pop = top map ((_, CoStackImpl(list.tail)))
    override def push[R >: T](r: R) = CoStackImpl(r :: list)
    override def toList = list


object TryCoStacks extends App:
  var stack: CoStack[Any] = CoStack[Int](10, 20, 30)
  stack = stack push "a string"
  val list: List[Any] = stack.toList
  println(list)
```

Covariant lists: CoList[+T]

```
// alternative approach: a sealead trait and case classes/objects
// sealed trait CoList[+T]
// case class Cons[+T](head: T, tail: CoList[T]) extends CoList[T]
// case object Nil extends CoList[Nothing]  // a single-object case

enum CoList[+T]:
  case Cons(head: T, tail: CoList[T])
  case Nil

object CoList:
  def apply[T](elems: T*): CoList[T] =
    elems.foldRight[CoList[T]](Nil)(Cons(_, _)) // functional trick

object TryCoList extends App:

  // would not be typed with CoList[T] instead of CoList[+T]
  val ll: CoList[CoList[Any]] = CoList(
    CoList(10,20,30),
    CoList("a", "b", "c"),
    CoList(10, new java.util.Date, "a string"))
  println(ll)
```

Outline

1. Aliases, abstract types, and abstract modelling
2. Traits and mixins
3. Exports and delegation vs inheritance
4. Variance and generics
5. Extension methods
6. Contextual programming with given instances
7. ScalaTest
8. For comprehension

Making the language more "flexible"

A trend in PLs

- programming languages are shifting towards less rigidity

⇒ to expand their application context of applications (DSLs, scripting), and to address agility more than predictivity

- programming languages are shifting towards higher and higher terseness

⇒ verbosity tends to hamper creativity

Features of Scala so far

- method/class names by symbols
- infix vs OOP unary method calls
- default/named arguments
- optional brackets
- powerful imports/exports

Features of Scala to rediscuss, all actually FP-oriented

- extension methods: enriching existing interfaces
- given instances: contextual programming
- automatic conversions: easing portability

⇒ use with care: be sure you correctly capture and document the abstraction!

Extension methods

An application of the adapter pattern

- there is a need of changing an interface after its initial definition, only in certain programming contexts (an interesting tool for agility)
- typically, adding new methods
- by the adapter pattern you define a new class whose objects have the new method and delegate to the "adaptee"

Extension methods: technique

- a method of a class defined externally to a class
- when the method is in scope, it can be called using dot notation on the receiver
- an extension method translates to a method with the "receiver" as the first argument

Extension methods: applications

- often used to add operators to existing classes
- often used to separate data from operations and recombine flexibly
- often used to give OOP flavour on top of pure FP

Example extension methods: it's pure FP!

```scala
case class Point2D(x: Double, y: Double)

extension (p: Point2D)
  def length: Double = Math.sqrt(p.x * p.x + p.y * p.y)
  def isZero: Boolean = p.x == 0.0 && p.y == 0

object Point2DOperators:
  extension (p: Point2D)
    def +(q: Point2D): Point2D = Point2D(p.x + q.x, p.y + q.y)
    def -(q: Point2D): Point2D = Point2D(p.x - q.x, p.y - q.y)

@main def main() =
  // using methods, defined separate to implementation
  val p = Point2D(10, 20)
  println(p.length)
  println(p.isZero)

  // importing and using a desired set of additional operators
  import Point2DOperators.*
  println(p + p)
  println(p - p)
  println((p - p).isZero)

  // extending and using a library class
  extension [T](l: List[T])
    def second: T = l.tail.head

  println(List(10,20,30).second)
```

Powerful-FP lists: do we really need OOP?

```scala
object OOPFPLists extends App:

  enum MyList[+A]:
    case Cons(h: A, t: MyList[A])
    case Empty

  object MyList:
    import MyList.*
    def apply[A](a: A*) = a.foldRight[MyList[A]](Empty)(Cons(_, _))
    extension [A](list: MyList[A])
      def foldRight[B](init: =>B)(op: (A, B) => B): B = list match
        case Empty => init
        case Cons(h, t) => op(h, t.foldRight(init)(op))
      def foldLeft[B](init: =>B)(op: (B, A) => B): B = list match
        case Empty => init
        case Cons(h, t) => t.foldLeft(op(init, h))(op)
      def map[B](f: A => B): MyList[B] =
        list.foldRight[MyList[B]](Empty)((a, b) => Cons(f(a), b))
      def filter(p: A => Boolean) =
        list.foldRight[MyList[A]](Empty)((a, b) => if p(a) then Cons(a, b) else b)
      def reduce(op: (A, A) => A): A = list match
        case Cons(h, t) => t.foldLeft(h)(op)

  import MyList.*
  val l = MyList(10, 20, 30, 40, 50)
  println(l map (_ - 1) filter (_ < 30) reduce (_ + _)) //57
```

Do we really need OOP?

Selectively getting rid of OOP

- define types by aliases, case classes, enums
- add OOP-like operations as extension methods in companion objects
- traits implementation is always private, mostly done by product types
- always rely on delegation instead of inheritance
- receiver/method is the same as "contextual argument"/pure-function

Pros

- suprisingly, it works very well in a huge number of cases
- if it works, it leads to very solid and controlled solutions: use it in Scala!
- type classes, e.g. monads, builds additional power on top

Cons

- sometimes, FP is too much of a stretching
- practically, in some cases OOP is just the right solution (controllers, stateful models)

Outline

1. Aliases, abstract types, and abstract modelling
2. Traits and mixins
3. Exports and delegation vs inheritance
4. Variance and generics
5. Extension methods
6. Contextual programming with given instances
7. ScalaTest
8. For comprehension

Contextual programming and abstractions

Generalising functions

- functions define computations as transformation of arguments (0, 1, or more) into outputs
- however, arguments can be often classified into two sorts:
  - inputs, naturally seen as the data that gets transformed into the outputs
  - context, that is used to affect how outputs are produced from inputs
- note this classification is neither universal nor the only possible one, but still it is quite recurring
- which implication?
  - context may/should be passed to the function using different mechanisms, e.g., it sometimes can be inferred from the scope of the call, without requiring boring/verbose explicit denotation

Term inference

- generally, the idea that in certain scopes the program can instruct the compiler on how to synthesise a "canonical" term for a given type
- Scala 2: the `implicit` mechanism – quite a criticised one, due to opacity
- Scala 3: the `given`/`using` mechanism – apparently more direct and intent-revealing

Using term inference: automatic strategies/configurations

Given instances

- a function/class formal parameter can be declared by a `using` modifier
- you can pass an argument as usual (with additional keyword `using`)
- but you can also skip it entirely: the compiler should be able to find a `given` instance defined elsewhere, to pass there
- very helpful since, in FP, methods tend to be highly configurable, often in tricky but useful ways

Mechanism

- `def f(using context: Context)(input: Input): Output = ???`
- explicit call: `f(using context)(input)`
- implicit call: `f(input)`
- definition of a canonical term in scope: `given input: Input = ???`
- importing givens: `import <object or package>.given`

Given parameters

```scala
object Parameters extends App:

  trait MyOrdering[A]:
    def greater(x: A, y: A): Boolean

  def max[T](elem: T, elems: T*)(using ordering: MyOrdering[T]): T =
    elems.foldLeft(elem)((m, e) => if ordering.greater(e,m) then e else m)

  val myStringOrdering = new MyOrdering[String]:
    override def greater(x: String, y: String) = x.compareTo(y) > 0

  println(max("a","b")(using myStringOrdering)) // "b", using a provided ordering

  // I want to use givens, with different styles of definition

  given implicitOrdering: MyOrdering[String] = myStringOrdering
  // without alias would shrink to: given MyOrdering[String] = ...
  // with in-line implementation of the trait with...
  given MyOrdering[Int] with
    def greater(x: Int, y: Int): Boolean = x > y
  // with functional interface to literal abbreviation
  given MyOrdering[List[Int]] = _.size > _.size

  // recovering canonical instance of MyOrdering[String]
  println(summon[MyOrdering[String]])

  println(max("a","b")) // "b", using the given ordering available in scope
  println(max(10, 20)) // 20, using the given ordering available in scope
  println(max(List(10,20), List(30))) // 20, using the given ordering available in scope
```

Context bounds and ad-hoc polymorphism

Avoiding given arguments with generic types

- "`def m[A: B](...)`" can be invoked with a type `A` only if there exists in scope a given value (strategy) of type `B[A]`
- `B` is called a context bound for `A`
- it means that `A` should have a "context" of type `B` that describes additional behaviour for it
- for instance a context bound `T:Ordering` means that when trying to instantiate `T` to `Int` we must be sure that a strategy `Ordering[Int]` can be inferred in scope
- when the given value is needed in the method this can be recovered with "`summon[B[A]]`"

Why? It enables ad-hoc polymorphism

- not a trick: it paves the way towards designing hierarchies of "contexts" as "type classes"
- a method becomes applicable to few selected types (ad-hoc), those with define context bound
- advanced FP uses these mechanisms to rely on different ways of "encapsulating"

Context bounds at play

```scala
object Parameters2 extends App:

  trait MyOrdering[A]:
    def greater(x: A, y: A): Boolean

  // equivalent to previous formulation, using a env bound
  def max[T: MyOrdering](elem: T, elems: T*): T =
    elems.foldLeft(elem)((m, e) => if summon[MyOrdering[T]].greater(e,m) then e else m)


  object GivenOrderings: // all shrinkable to literals
    given MyOrdering[String] with
      override def greater(x: String, y: String) = x.compareTo(y) >= 0
    given MyOrdering[Int] with
      override def greater(x: Int, y: Int) = x>=y
    // a generic way of expressing given terms, alias needed
    given listOrdering[A]: MyOrdering[List[A]] with
      override def greater(x: List[A], y: List[A]) = x.size >= y.size

  import GivenOrderings.given

  println(max("a","d","b","c")) // "d"
  println(max(10,40,20,30)) // 40
  println(max(List(10,20),List(30,40,50),List("a","b"))) // List(30,40,50)
```

Implicit Conversions

The need of automatic adapters

- extension methods provide a nice way of adding methods to classes that do not have them at definition time
- still there are many contexts in which we have legacy libraries and want them to interoperate to new ones with different assumptions
- the adapter pattern can be used
  - you essentially write a wrapper class
  - you wrap legacy code, the wrapper has the new desired interface
- this pattern however adds considerable boilerplate code: we need automatisation

Implicit/given conversions in Scala 3

- you provide given instances of type `scala.Conversion[-I,+O]` (... `extends` `I => O`)

Implicit Conversions

```
object Conversions extends App:
  // an example class and a conversion defined in its companion
  case class Point(n: Int, m: Int)
  object Point: // the companion object is a proper place for conversions
    // specifying a conversion from tuple to point, by apply method
    given Conversion[(Int, Int), Point] with
      def apply(t: (Int,Int)): Point =
        println("converting tuple " + t + " to a point..") // a debug print
        Point(t._1, t._2)

    // specifying a conversion by a literal
    given Conversion[List[Int], Point] = l => Point(l.head, l.tail.head)

  def pointToRadius(p: Point) = Math.sqrt(p.n * p.n + p.m * p.m)

  // the given conversion is already visible as in Point companion
  println(pointToRadius(Point(3, 4))) // 5

  val tup = (3,4)
  println(pointToRadius(tup)) // 5
  println(pointToRadius(List(3,4))) // 5
  // try desugaring with Ctrl-Alt-D in IntelliJ

  // a facility to check (e.g. in REPL) if a given conversion is active
  println(summon[((Int,Int)) => Point]( (10,20) )) //Point(10,20)
  println(summon[List[Int] => Point]( List(20,30,40) )) //Point(10,20)
  println(summon[Nothing => Point]( List(20,30,40) )) //Point(10,20)
```

An example case, facading a legacy library

```scala
import java.awt.event.*
import javax.swing.*

object ScalaSwingFacade:

  def createPanel(comps: JComponent*): JPanel =
    comps.foldLeft(new JPanel()): (p, c) =>
      p.add(c); p

  given Conversion[ActionEvent => Unit, ActionListener] = f => f(_)
    // f => new ActionListener(){ def actionPerformed(e: ActionEvent) = f(e) }
    // f => e => f(e)

  extension (b: JButton)
    def addActionListener(action: => Unit): Unit = addActionListener(e => action)

object TrySwing extends App:
  import ScalaSwingFacade.{*, given}

  val f = new JFrame("title")
  val text = new JTextField(10)
  val bok = new JButton("OK")
  val bevt = new JButton("Event")
  val bquit = new JButton("Quit")
  val p = createPanel(text, bok, bevt, bquit)
  bok.addActionListener(JOptionPane.showMessageDialog(f,s"Text: ${text.getText}"))
  bevt.addActionListener(JOptionPane.showMessageDialog(f, _))
  bquit.addActionListener(System.exit(0))
  f.getContentPane.add(p)
```

Outline

1. Aliases, abstract types, and abstract modelling
2. Traits and mixins
3. Exports and delegation vs inheritance
4. Variance and generics
5. Extension methods
6. Contextual programming with given instances
7. ScalaTest
8. For comprehension

An example use of pretty DSL / library pimping: ScalaTest

```scala
// libraryDependencies += "org.scalatest" %% "scalatest" % "3.2.18" % "test"
// check details at: https://www.scalatest.org/

import org.scalatest.matchers.should.Matchers.*

class ScalaTestShowcase extends org.scalatest.funsuite.AnyFunSuite:

  test("List operations, and should matchers"):
    val testedList: List[Int] = 10 :: 20 :: 30 :: 40 :: Nil
    assert(testedList.size == 4) // JUnit-like assertion
    testedList.size shouldBe 4   // from here, should matchers...
    testedList.size should be (4)
    testedList should have size 4
    testedList should not be empty
    testedList.forall(_ > 5) should be (true)
    a [IndexOutOfBoundsException] should be thrownBy testedList(5)
    testedList.collect{case i if i > 20 => i + 1} should contain allOf (31, 41)
    "val l = 10 :: 20 :: Nil" should compile

class SetSpec extends org.scalatest.flatspec.AnyFlatSpec:

  "A mutable Set" should "correctly add elements" in:
    val s: collection.mutable.Set[Int] = collection.mutable.Set()
    s += 10
    s += 20
    s += 30
    assert(s == Set(10, 20, 30))
    s should be (Set(10, 20, 30))
```

A creative use of DSLs in tests: https://github.com/jahrim/PPS-22-chess

```
"A chess board at the beginning of a chess game"
    should "abide to the standard configuration for the chess pieces" in:

    ChessBoard.standard shouldEqual ChessBoard:
      r | n | b | q | k | b | n | r
      p | p | p | p | p | p | p | p
      * | * | * | * | * | * | * | *
      * | * | * | * | * | * | * | *
      * | * | * | * | * | * | * | *
      * | * | * | * | * | * | * | *
      P | P | P | P | P | P | P | P
      R | N | B | Q | K | B | N | R
```

Outline

1. Aliases, abstract types, and abstract modelling
2. Traits and mixins
3. Exports and delegation vs inheritance
4. Variance and generics
5. Extension methods
6. Contextual programming with given instances
7. ScalaTest
8. For comprehension

For comprehension

Weak understanding

- a powerful iteration-like construct
- works with more variables
- somewhat equivalent to use `foreach`, perhaps `map/flatmap`?

Stronger understanding (only one further step here. . . , more to come)

- a key mechanism to encode exploration-like computations
- a key mechanism to define a notion of "sequentiality" in FP
- a key mechanism to encapsulate monads

For now

- `for/yield` construct is a sugar for use of `map/flatMap/filter`

Desugaring for comprehension

```scala
object Comprehension extends App:

  val l = List(10,20,30)
  println(l map (_ * 2))             // 20,40,60
  println(l flatMap (List(_, 0)))  // 10,0,20,0,30,0
  println(l filter (_ > 10))         // 20,30

  // Problem: give all quartets (x,y,z,x*y*z) such that:
  // 1) x,y,z, in 1..9; 2) the sum of x,y,z gives 10; 3) x < y < z

  val l2 = for
    x <- 1 until 9
    y <- 1 until 9
    z <- 1 until 9
    if x + y + z == 10
    if x < y && y < z
    w = x * y * z
  yield
    (x, y, z, w)
  println(l2) // Vector((1,2,7,14), (1,3,6,18), (1,4,5,20), (2,3,5,30))

  // desugared version (Scala 2)
  val l3 = (1 until 9)
    .flatMap( x => (1 until 9)
      .flatMap( y => (1 until 9)
        .filter( z => x + y + z == 10 )
        .filter( z => x < y && y < z )
        .map( z => (z, x * y * z))
        .map{ case (z, w) => (x, y, z, w)}))
  println(l3) // Vector((1,2,7,14), (1,3,6,18), (1,4,5,20), (2,3,5,30))
```

Exloring search spaces: the n queens problem

```scala
object Nqueen:
  type Queen = (Int, Int) // Type aliases improve readability
  type Solution = Iterable[Queen]
  type IterableFactory = Solution => Iterable[Solution]
  val size = 10
  given IterableFactory = LazyList(_) // could switch to Set(_), List(_), or List(_).view

  @main def run() = placeQueens().zipWithIndex foreach (printSolution(_))

  def placeQueens(n: Int = size)(using factory: IterableFactory): Iterable[Solution] = n match
    case 0 => factory(Set())
    case _ =>
      for
        queens <- placeQueens(n - 1)  // get a solution on a smaller grid
        y <- 1 to size                // iterate over all columns
        queen = (n, y)                // consider a new queen on any cell in last row
        if isSafe(queen, queens)      // ...only a safe one
      yield
        queens.toSeq :+ queen                // construct the new solution

  def isSafe(queen: Queen, others: Iterable[Queen]) =
    others forall (!isAttacked(queen, _))

  def isAttacked(q1: Queen, q2: Queen) =
    q1._1 == q2._1 || q1._2 == q2._2 || (q2._1 - q1._1).abs == (q2._2 - q1._2).abs

  def printSolution(si: (Solution, Int)): Unit =
    println(); println(s"sol ${si._2}")
    for queen <- si._1; x <- 1 to size do
      print(if queen._2 == x then "Q " else ". ")
      if x == size then println()
```